🔬 Building a Clinical-Grade Breast Cancer Predictor

A Beginner's Technical Walkthrough

Welcome! If you are reading this, you want to understand how to build a production-ready Machine Learning system from scratch. This guide breaks down the architecture, the code, and the rationale behind the **Breast Cancer Predictor** project.

---

1. Project Overview & Data 📊

The Objective

In healthcare, a **False Negative** (telling a patient they don't have cancer when they actually do) is the worst possible outcome. Our goal was to build a binary classification system that takes in 30 physical features of a breast tumour and predicts if it is `Malignant` (cancerous) or `Benign` (non-cancerous). We programmed the system specifically to **maximise sensitivity** (avoiding missed cancers).

The Dataset

We used the **Wisconsin Breast Cancer Diagnostic (WBCD)** dataset. It contains 569 patient samples. Real doctors took Fine Needle Aspirates (FNA)—a type of biopsy—and digitised the images. They extracted 10 core metrics about the cell nuclei:

- Radius, Texture, Perimeter, Area, Smoothness, Compactness, Concavity, Concave Points, Symmetry, Fractal Dimension. For each metric, they recorded the **Mean**, **Standard Error**, and **Worst (Max)** value, resulting in `10 x 3 = 30` input features.

---

2. Code Architecture & Folders 🗂️

We structured the codebase professionally (like top tech companies do), separating concerns into logical units:

- `src/data/`: Handles loading the raw CSV, exploring the data (EDA), and cleaning (preprocessing).
- `src/models/`: Contains the logic for training algorithms and searching for the best hyperparameters.
- `src/evaluate/`: Calculates clinical metrics (AUC-ROC, Sensitivity) and explains the AI's "thoughts" using SHAP.
- `src/api/`: The Web Server (FastAPI) and Authentication layer.
- `dashboard/`: The Front-End User Interface (Streamlit) for clinicians.
- `config/config.yaml`: A central hub to tweak numbers without touching core code.

---

3. Step-by-Step Pipeline Explanation 🚀

Step A: Data Preprocessing (`src/data/preprocessor.py`)

Machine Learning models hate missing data and wildly different number scales (e.g., comparing an `area` of 1000 to a `smoothness` of 0.01). Our pipeline does three things automatically:

1. **SimpleImputer:** Fills any missing data with the median value of that column.
2. **HighCorrelationDropper:** If two columns are mathematically identical (like `radius` and `perimeter`, which are heavily correlated), dropping one speeds up training and removes "noise" without losing information.
3. **StandardScaler:** Squishes all values so they have a mean of 0 and a standard deviation of 1.

Step B: Training Multiple Models (`src/models/trainer.py`)

We didn't just pick one algorithm; we held a competition! We trained:

- **Logistic Regression:** A classic statistical model.
- **Random Forest:** Hundreds of decision trees voting together.
- **XGBoost:** A highly advanced gradient-boosting tree algorithm.
- **Support Vector Machine (SVM):** Finds the optimal geometric line separating classes.
- **Multi-Layer Perceptron (MLP):** A mini neural network mirroring the human brain.

We used **GridSearchCV (5-fold Cross-Validation)** to test hundreds of configurations. It splits the data into 5 chunks, rotating which chunk it uses as a 'test' to ensure the AI isn't just memorizing the answers (overfitting).

Step C: Clinical Evaluation (`src/evaluate/metrics.py`)

Once trained, we put the models through a clinical test.

- **Sensitivity (Recall):** Out of all actual Malignant cases, how many did the AI catch?
- **Specificity:** Out of all actual Benign cases, how many did it correctly clear? We selected **Random Forest**, which achieved a **Sensitivity of ~98.15%** and an **AUC-ROC of ~0.995**. We achieved this by optimising the **Youden Index**, which mathematically shifts the "confidence threshold" heavily towards preventing False Negatives.

Step D: SHAP Explainability (`src/evaluate/explainer.py`)

Doctors cannot trust a "black box" that just spits out answers. We integrated **SHAP (SHapley Additive exPlanations)**. Derived from Game Theory, SHAP breaks down exactly how much each of the 30 features pushed the final prediction up or down. For example, if `worst_concave_points` is very high, SHAP will show exactly how many percentage points that specific feature added to the cancer probability.

Step E: Real-world Deployment (`src/api/main.py` & `dashboard/app.py`)

Models on laptops don't help patients.

1. We wrapped the model in **FastAPI**, creating a lightning-fast REST Web Server. Other hospital software can securely ping this API with patient data and get back an instant prediction.

2. We built a visual **Streamlit Dashboard**. Instead of writing code, a clinician opens a beautiful web page, inputs numbers into sliders, and sees clear red/green results and SHAP charts immediately.

Step F: Continuous Integration & Containerisation 🧪

Finally, we ensured the system was indestructible:

- **Pytest:** We wrote automated tests (`tests/test_api.py`) to verify the API rejects bad data.
- **Docker:** We packaged the entire codebase, Python version, and libraries into isolated "Containers". This ensures the software runs flawlessly on an AWS cloud server identically to how it runs on your Macbook.
- **GitHub Actions:** A pipeline (`ml_pipeline.yml`) that auto-runs Pytest and Linting every time code is updated, preventing broken updates from entering production.

🏆 Summary

This project represents a complete, end-to-end Machine Learning lifecycle. We transformed raw hospital CSV data into a mathematically rigorous, mathematically explainable, continuously-tested, securely-authenticated web application ready for cloud deployment.